LEVEL 2 HACKATHON

FUNDAMENTALS OF WEB DEVELOPMENT

STUDENT ATTENDANCE TRACKER

Project Documentation

Student Details	Information
Student Name	S. Rokhaiya
Register Number	25131111802522071
Department	BSc. CS
ID	asanm25131111802522071
College	Krishnasamy College of Science, Arts and Management for Women
Date of Submission	26 October 2026
GitHub Link	

Prepared using the submitted Student Attendance Tracker HTML file and the supplied Level 2 Hackathon template. The GitHub link was not included in the supplied information, so it is marked as not provided.

1. Problem Statement

Maintaining student attendance manually can take time and make it difficult to find a student's attendance history or calculate attendance percentages. Paper records can also be difficult to search, update, and summarize consistently.

The Student Attendance Tracker is a browser-based web application that helps a user maintain student records, mark daily attendance, review attendance percentages, filter records, and view each student's attendance history. It is intended to make basic attendance management clearer and faster.

2. Objectives

• Add, edit, and delete student records.

• Record attendance for a selected date using Present, Absent, or Leave.

• Display summary counts for total students, present today, absent today, and overall attendance.

• Calculate each student's attendance percentage using Present days divided by Present plus Absent days.

• Search by student name, ID, or department and filter by attendance status.

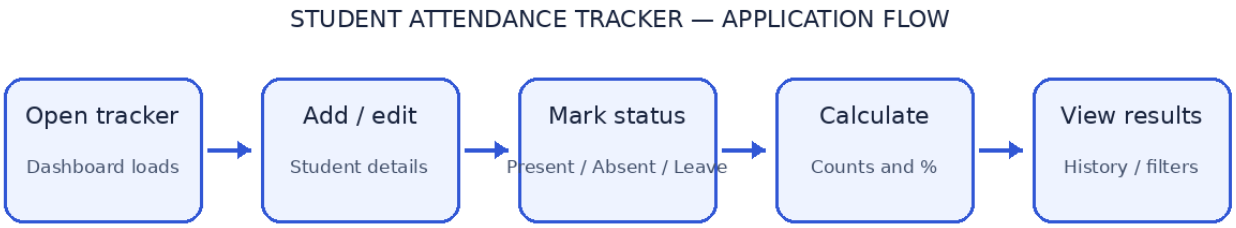
• Warn when a student's attendance percentage is below a configurable threshold.

• Store records in the browser using localStorage so they remain available in the same browser.

• Show attendance history for a selected student.

3. Architecture

The application is a client-side web page. HTML defines the dashboard, student form, summary cards, attendance table, filters, and history area. CSS controls the layout, colors, cards, table, progress bars, and responsive behavior. JavaScript handles form validation, student record operations, attendance status changes, calculations, searching, filtering, history display, and saving/loading browser data.



User navigation flow: open the tracker → review dashboard → add or edit a student → choose the attendance date and mark a status → review summary and percentage → search/filter records or open attendance history. The app reads and writes records through the browser's localStorage; no backend server or external API is included in the supplied code.

4. Practical Implementation

The supplied implementation places HTML, CSS, and JavaScript in a single .html file. Open that file in a modern browser to run it. The application initializes the selected date, loads saved records, attaches event listeners, and renders the dashboard. When a record changes, the application saves the data and refreshes the displayed counts and table.

4.1 Features Implemented

Student management	Add a student with name, student ID/roll number, and department; edit existing details; delete a student after confirmation.
Validation	Required fields must be filled. Student IDs are checked without case sensitivity to prevent duplicates.
Date-based attendance	Choose a date and mark each student Present, Absent, Leave, or Not marked.
Dashboard summary	Shows total students, present today, absent today, and overall attendance percentage.
Attendance percentage	Present ÷ (Present + Absent) × 100; Leave and unmarked days are excluded.
Search and filter	Searches name, ID, or department and filters by selected-date status.
Low attendance alert	Flags students whose calculated percentage is below the configured threshold.
Attendance history	Displays saved date/status entries for the selected student.
Browser storage	Uses localStorage to keep data in the same browser on the same device.

4.2 Source Code

The following code is taken from the submitted HTML file. It is split into HTML, CSS, and JavaScript sections for readability. The original application is a single-file implementation.

HTML

HTML structures the page into a header with the attendance date, summary cards, an add/edit student form, search and status filters, the student attendance table, an attendance history panel, and a footer. Element IDs allow JavaScript to update the interface.

```
&lt;body&gt;
&lt;header&gt;
  &lt;div&gt;&lt;h1&gt;Student Attendance Tracker&lt;/h1&gt;&lt;p&gt;Manage student records and daily attendance&lt;/p&gt;&lt;/div&gt;
  &lt;div&gt;&lt;label for="attendanceDate" style="color:white"&gt;Attendance date&lt;/label&gt;&lt;input type="date" id="attendanceDate"&gt;&lt;
&lt;/header&gt;
&lt;main&gt;
  &lt;section class="cards"&gt;
    &lt;div class="card"&gt;&lt;span&gt;Total students&lt;/span&gt;&lt;strong id="totalCount"&gt;0&lt;/strong&gt;&lt;/div&gt;
    &lt;div class="card"&gt;&lt;span&gt;Present today&lt;/span&gt;&lt;strong id="presentCount"&gt;0&lt;/strong&gt;&lt;/div&gt;
    &lt;div class="card"&gt;&lt;span&gt;Absent today&lt;/span&gt;&lt;strong id="absentCount"&gt;0&lt;/strong&gt;&lt;/div&gt;
    &lt;div class="card"&gt;&lt;span&gt;Overall attendance&lt;/span&gt;&lt;strong id="averageCount"&gt;0%&lt;/strong&gt;&lt;/div&gt;
  &lt;/section&gt;
  &lt;section class="panel"&gt;
    &lt;h2 id="formTitle"&gt;Add student&lt;/h2&gt;
    &lt;form id="studentForm"&gt;
      &lt;div class="form-grid"&gt;
        &lt;div&gt;&lt;label for="studentName"&gt;Student name&lt;/label&gt;&lt;input id="studentName" required maxlength="70" placeholder="e.g. A
        &lt;div&gt;&lt;label for="studentRoll"&gt;Student ID / Roll number&lt;/label&gt;&lt;input id="studentRoll" required maxlength="25" placeho
        &lt;div&gt;&lt;label for="studentClass"&gt;Department&lt;/label&gt;&lt;input id="studentClass" required maxlength="50" placeholder="e.g. B
      &lt;/div&gt;
      &lt;div class="actions"&gt;&lt;button type="submit" id="saveButton"&gt;Add student&lt;/button&gt;&lt;button type="button" class="secondary"
    &lt;/form&gt;
  &lt;/section&gt;
```

```
  &lt;section class="panel"&gt;
    &lt;h2&gt;Student attendance&lt;/h2&gt;
    &lt;div id="lowNotice" class="notice"&gt;&lt;/div&gt;
    &lt;div class="form-grid" style="margin-bottom:14px"&gt;
      &lt;div&gt;&lt;label for="searchBox"&gt;Search by name, student ID, or department&lt;/label&gt;&lt;input id="searchBox" placeholder="Start
      &lt;div&gt;&lt;label for="statusFilter"&gt;Filter by selected-date status&lt;/label&gt;&lt;select id="statusFilter"&gt;&lt;option value="Al
      &lt;div&gt;&lt;label for="threshold"&gt;Low attendance threshold (%)&lt;/label&gt;&lt;input id="threshold" type="number" min="1" max="100"
    &lt;/div&gt;
    &lt;div class="table-wrap"&gt;&lt;table&gt;
      &lt;thead&gt;&lt;tr&gt;&lt;th&gt;Student&lt;/th&gt;&lt;th&gt;Student ID&lt;/th&gt;&lt;th&gt;Attendance %&lt;/th&gt;&lt;th&gt;Selected date
      &lt;tbody id="studentRows"&gt;&lt;/tbody&gt;
    &lt;/table&gt;&lt;/div&gt;
    &lt;p class="muted"&gt;Attendance percentage = Present days ÷ (Present + Absent days) × 100. Leave and unmarked days are excluded.&lt;/p&gt;
  &lt;/section&gt;
  &lt;section class="panel"&gt;
    &lt;h2&gt;Attendance history&lt;/h2&gt;
    &lt;p class="muted"&gt;Select “View history” for a student to see their attendance history.&lt;/p&gt;
    &lt;div id="historyBox" class="muted"&gt;No student selected.&lt;/div&gt;
  &lt;/section&gt;
  &lt;footer&gt;Student Attendance Tracker · Records are saved in this browser using local storage.&lt;/footer&gt;
&lt;/main&gt;
&lt;/body&gt;
```

CSS

CSS styles the interface using a blue theme, responsive grid layouts, cards, form controls, status badges, progress bars, and a mobile layout for narrow screens.

```
&lt;style&gt;
:root{--blue:#3157d5;--bg:#f4f7fc;--ink:#18233d;--muted:#68738c}
*{box-sizing:border-box}
body{margin:0;font-family:Arial,sans-serif;background:var(--bg);color:var(--ink)}
header{background:linear-gradient(120deg,#243c9b,#4777ef);color:#fff;padding:24px 5%;display:flex;justify-content:space-between;align-items:cente
header h1{margin:0;font-size:clamp(22px,3vw,30px)}header p{margin:7px 0 0;color:#e3eaff}
main{max-width:1150px;margin:24px auto;padding:0 18px}
.cards{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:14px;margin-bottom:20px}
.card,.panel{background:#fff;border:1px solid #e4e9f3;border-radius:14px;padding:18px;box-shadow:0 5px 18px #20376b0a}
.card span{font-size:13px;color:var(--muted)}.card strong{display:block;font-size:29px;margin-top:8px}
.panel{margin-bottom:18px}.panel h2{font-size:18px;margin:0 0 15px}
.form-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(180px,1fr));gap:12px}
label{display:block;font-size:13px;font-weight:bold;margin-bottom:6px;color:#48536b}
input,select{width:100%;padding:11px;border:1px solid #d5dbea;border-radius:8px;font:inherit;background:#fff}
button{border:0;border-radius:8px;padding:10px 14px;background:var(--blue);color:#fff;font-weight:bold;cursor:pointer}
button:hover{filter:brightness(.93)}button.secondary{background:#e9edfb;color:#2947ad}button.danger{background:#fff0f0;color:#b42323}
.actions{display:flex;gap:8px;flex-wrap:wrap;margin-top:13px}
.table-wrap{overflow-x:auto}table{border-collapse:collapse;width:100%;min-width:720px}
th,td{text-align:left;padding:12px;border-bottom:1px solid #edf0f6;font-size:14px}th{background:#f7f9fe;color:#5c6780;font-size:12px;text-transfo
.pill{display:inline-block;padding:5px 8px;border-radius:20px;font-size:12px;font-weight:bold}.good{background:#e5f7ee;color:#13744d}.low{backgro
.muted{color:#6a748a;font-size:13px}.notice{padding:12px;border-radius:9px;background:#fff3df;color:#8b4a00;margin-bottom:12px;display:none}
.progress{height:7px;background:#e9edf6;border-radius:8px;overflow:hidden;margin-top:6px}.progress div{height:100%;background:#4777ef}
footer{text-align:center;color:#7a8499;font-size:12px;padding:20px}
@media(max-width:600px){header{padding:20px}main{margin-top:14px;padding:0 12px}.panel,.card{padding:14px}}
&lt;/style&gt;
```

JavaScript

JavaScript loads and saves records, validates the student form, prevents duplicate IDs, manages attendance statuses, calculates percentages, builds the filtered table, shows attendance history, and refreshes the dashboard.

```
&lt;script&gt;
const $ = id =&gt; document.getElementById(id);
const today = () =&gt; { const d=new Date(); return `${d.getFullYear()}-${String(d.getMonth()+1).padStart(2,'0')}-${String(d.getDate()).padStart(
$('attendanceDate').value = today();
let students = [];
try { students = JSON.parse(localStorage.getItem('attendanceStudentsV1') || '[]'); } catch(e) { students=[]; }
let editingId = null, historyStudentId = null;
const statuses = ['Present','Absent','Leave'];
function saveData(){ localStorage.setItem('attendanceStudentsV1', JSON.stringify(students)); }
function selectedDate(){ return $('attendanceDate').value || today(); }
function recordFor(student,date){ return (student.attendance||{})[date] || ''; }
function counts(student){
  const values=Object.values(student.attendance||{});
  const present=values.filter(v=&gt;v==='Present').length, absent=values.filter(v=&gt;v==='Absent').length, counted=present+absent;
  return {present,absent,counted,percent:counted?Math.round(present/counted*100):0};
}
function setStatus(id,status){
  const student=students.find(s=&gt;s.id===id); if(!student)return;
  const date=selectedDate();
  if(status) student.attendance[date]=status; else delete student.attendance[date];
  saveData(); render();
}
function resetForm(){editingId=null;$('studentForm').reset();$('formTitle').textContent='Add student';$('saveButton').textContent='Add student';$
$('studentForm').addEventListener('submit',e=&gt;{
  e.preventDefault();
  const name=$('studentName').value.trim(), roll=$('studentRoll').value.trim(), group=$('studentClass').value.trim();
  if(!name||!roll||!group)return;
  const duplicate=students.some(s=&gt;s.roll.toLowerCase()===roll.toLowerCase()&amp;&amp;s.id!==editingId);
  if(duplicate){alert('That student ID already exists. Please enter a unique ID.');return;}
```

```
  if(editingId){const s=students.find(x=&gt;x.id===editingId);if(s)Object.assign(s,{name,roll,group});}
  else students.push({id:Date.now().toString()+Math.random().toString(16).slice(2),name,roll,group,attendance:{}});
  saveData();resetForm();render();
});
$('cancelEdit').addEventListener('click',resetForm);
function editStudent(id){
  const s=students.find(x=&gt;x.id===id);if(!s)return;editingId=id;
  $('studentName').value=s.name;$('studentRoll').value=s.roll;$('studentClass').value=s.group;
  $('formTitle').textContent='Edit student';$('saveButton').textContent='Save changes';$('cancelEdit').hidden=false;$('studentName').focus();
}
function deleteStudent(id){
  const s=students.find(x=&gt;x.id===id);if(!s)return;
  if(confirm('Delete '+s.name+' and all their attendance records?')){
    students=students.filter(x=&gt;x.id!==id);saveData();if(editingId===id)resetForm();if(historyStudentId===id)historyStudentId=null;render();
  }
}
function escapeHtml(value){return String(value).replace(/[&amp;&lt;&gt;"']/g,c=&gt;({'&amp;':'&amp;amp;','&lt;':'&amp;lt;','&gt;':'&amp;gt;','"':
function viewHistory(id){
  historyStudentId=id;const s=students.find(x=&gt;x.id===id);if(!s)return;
  const entries=Object.entries(s.attendance||{}).sort((a,b)=&gt;b[0].localeCompare(a[0]));
  $('historyBox').innerHTML='&lt;strong&gt;'+escapeHtml(s.name)+' ('+escapeHtml(s.roll)+')&lt;/strong&gt;&lt;br&gt;'+
    (entries.length?entries.map(([date,status])=&gt;escapeHtml(date)+' — '+escapeHtml(status)).join('&lt;br&gt;'):'No attendance records yet.');
}
function render(){
  const date=selectedDate(),query=$('searchBox').value.trim().toLowerCase(),filter=$('statusFilter').value;
  const threshold=Math.max(1,Math.min(100,Number($('threshold').value)||75));
  const presentToday=students.filter(s=&gt;recordFor(s,date)==='Present').length,absentToday=students.filter(s=&gt;recordFor(s,date)==='Absent').
  const allCounts=students.map(counts),counted=allCounts.reduce((n,c)=&gt;n+c.counted,0),totalPresent=allCounts.reduce((n,c)=&gt;n+c.present,0);
  $('totalCount').textContent=students.length;$('presentCount').textContent=presentToday;$('absentCount').textContent=absentToday;
  $('averageCount').textContent=(counted?Math.round(totalPresent/counted*100):0)+'%';
  const low=students.filter(s=&gt;counts(s).counted&gt;0&amp;&amp;counts(s).percent&lt;threshold);
  $('lowNotice').style.display=low.length?'block':'none';
  $('lowNotice').textContent=low.length?'Low attendance (below '+threshold+'%): '+low.map(s=&gt;s.name+' ('+counts(s).percent+'%)').join(', '):''
  const filtered=students.filter(s=&gt;{
    const status=recordFor(s,date)||'Not marked';
    return (s.name.toLowerCase().includes(query)||s.roll.toLowerCase().includes(query)||s.group.toLowerCase().includes(query))&amp;&amp;(filter===
  });
  $('studentRows').innerHTML=filtered.map(s=&gt;{
    const c=counts(s),status=recordFor(s,date),badge=c.counted&amp;&amp;c.percent&lt;threshold?'low':'good';
    return `&lt;tr&gt;
      &lt;td&gt;&lt;strong&gt;${escapeHtml(s.name)}&lt;/strong&gt;&lt;br&gt;&lt;span class="muted"&gt;${escapeHtml(s.group)}&lt;/span&gt;&lt;br&g
      &lt;td&gt;${escapeHtml(s.roll)}&lt;/td&gt;
      &lt;td&gt;&lt;strong&gt;${c.percent}%&lt;/strong&gt;&lt;div class="progress"&gt;&lt;div style="width:${c.percent}%"&gt;&lt;/div&gt;&lt;/div
      &lt;td&gt;&lt;select aria-label="Attendance status" onchange="setStatus('${s.id}',this.value)"&gt;&lt;option value=""&gt;Not marked&lt;/opt
      &lt;td&gt;&lt;div class="actions"&gt;&lt;button type="button" class="secondary" onclick="editStudent('${s.id}')"&gt;Edit&lt;/button&gt;&lt;
    &lt;/tr&gt;`;
  }).join('');
  if(!filtered.length)$('studentRows').innerHTML='&lt;tr&gt;&lt;td colspan="5" class="muted"&gt;No students found. Add a student above to get sta
  if(historyStudentId&amp;&amp;students.some(s=&gt;s.id===historyStudentId))viewHistory(historyStudentId);else $('historyBox').textContent='No st
}
$('attendanceDate').addEventListener('change',render);$('searchBox').addEventListener('input',render);
$('statusFilter').addEventListener('change',render);$('threshold').addEventListener('input',render);
render();
&lt;/script&gt;
```

4.3 Project Execution

• Save the supplied file as student_attendance_tracker.html.

• Open the file in Chrome, Edge, Firefox, or another modern browser. No installation or server is required for the supplied version.

• Enter the student name, student ID/roll number, and department, then select Add student.

• Choose the attendance date in the header. For each student, choose Present, Absent, Leave, or Not marked.

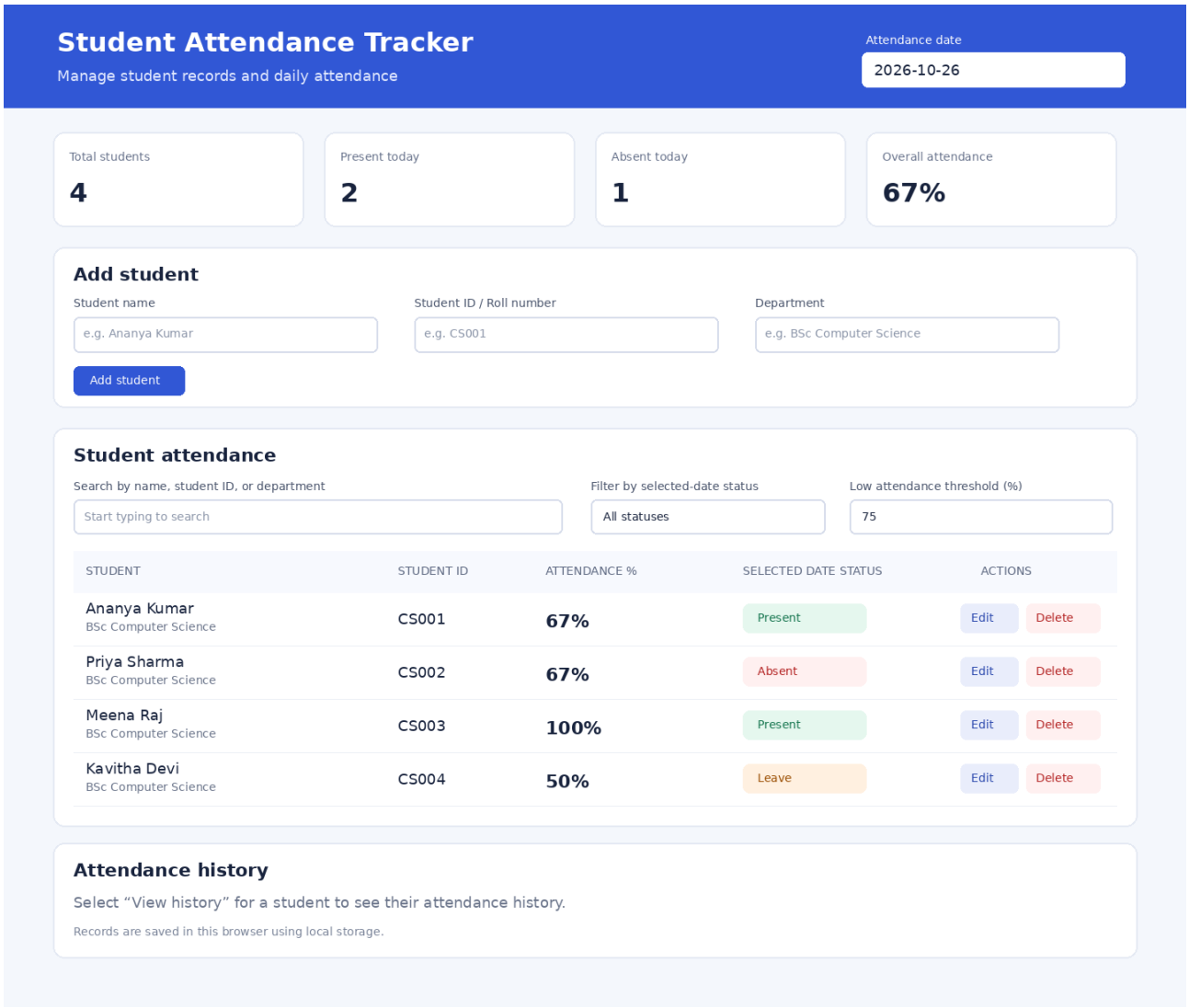
• Use the search field and status filter to narrow the list. Change the low-attendance threshold if required.

• Select View history to see a student’s recorded attendance. Use Edit to update details or Delete to remove a record.

• The application saves changes in localStorage automatically. Data is stored in that browser and is not automatically shared with other devices or users.

5. Output Screenshots

The screenshot below shows a demonstration of the submitted application using sample records. The sample entries are only for demonstrating the interface and are not real student records.



Expected visible output: summary cards at the top; student entry form; searchable attendance table with attendance percentages and status selectors; and an attendance history section. The displayed counts depend on the records and selected date in the browser.

6. Technical Explanation

• DOM selection: the helper function $(id) uses document.getElementById to locate page elements.

• Event handling: submit, click, input, and change listeners respond to user actions and call render() to refresh the interface.

• Data model: each student contains a unique internal id, name, roll number, department/group, and an attendance object keyed by date.

• Persistence: JSON.stringify converts the students array into text for localStorage; JSON.parse restores it when the page opens.

• Attendance calculation: counts() totals Present and Absent entries and computes the percentage only from those counted days.

• Safe display: escapeHtml() encodes HTML-sensitive characters before user-entered text is inserted into HTML strings.

• Responsive design: CSS Grid, flexible containers, and a media query adapt the layout to different screen widths.

• Limitations: the supplied version has no login, central database, cloud synchronization, or role-based access. Browser data can be lost if local browser storage is cleared.

7. Conclusion

The Student Attendance Tracker demonstrates the use of HTML, CSS, and JavaScript to build an interactive web application. It supports basic student record management, date-wise attendance marking, automatic percentage calculation, search and filtering, low-attendance alerts, and attendance history. The project also demonstrates form validation, DOM updates, event handling, and browser localStorage.

8. Future Enhancements

• Add user authentication and role-based access for faculty and administrators.

• Connect to a secure database and backend API so authorized users can access shared records across devices.

• Add export and print options for attendance reports in PDF or CSV format.

• Provide monthly summaries, charts, and date-range reports.

• Add backup and restore features and clearer handling for browser storage errors.

• Improve accessibility with stronger keyboard navigation, focus indicators, and screen-reader labels.

• Add automated tests and server-side validation if a backend is introduced.

Submission note: insert the actual GitHub repository URL if available before uploading. The screenshots in this PDF are demonstration output using sample data; replace or supplement them with screenshots from your own browser run if your evaluator requires direct evidence from your device.